Tactical Vehicle Simulation POC - Backend Documentation

Technology Stack

- Spring Boot 4.0.6
- Java 25
- Maven
- PostgreSQL
- Spring Data JPA
- Scheduler (@EnableScheduling)
- Lombok

Database

Create Database

```
CREATE DATABASE tactical_db
WITH
OWNER = postgres
ENCODING = 'UTF8';
```

Table: plan

```
CREATE TABLE plan
(
    id BIGSERIAL PRIMARY KEY,
    plan_name VARCHAR(100),
    speed DOUBLE PRECISION,
    start_time TIMESTAMP,
    status VARCHAR(50)
);
```

Table: route_point

```
CREATE TABLE route_point
(
    id BIGSERIAL PRIMARY KEY,
    plan_id BIGINT NOT NULL,
```

```
    sequence_no INT,
    latitude DOUBLE PRECISION,
    longitude DOUBLE PRECISION,

    CONSTRAINT fk_route_plan
    FOREIGN KEY(plan_id)
    REFERENCES plan(id)
);
```

Table: vehicle

```
CREATE TABLE vehicle
(
    id BIGSERIAL PRIMARY KEY,
    vehicle_name VARCHAR(100),
    vehicle_type VARCHAR(50),
    fuel_capacity DOUBLE PRECISION,
    fuel_remaining DOUBLE PRECISION,
    current_latitude DOUBLE PRECISION,
    current_longitude DOUBLE PRECISION
);
```

Additional Columns Required for Simulation

```
ALTER TABLE vehicle
ADD COLUMN current_plan_id BIGINT;

ALTER TABLE vehicle
ADD COLUMN current_point_index INT;

ALTER TABLE vehicle
ADD COLUMN simulation_started BOOLEAN;
```

Enums

VehicleType

```
public enum VehicleType {

    ARMY_VEHICLE,
    TANK,
```

```
    TRUCK
}
```

PlanStatus

```
public enum PlanStatus {

    CREATED,
    RUNNING,
    COMPLETED,
    STOPPED
}
```

DTOs

CoordinateDto

```
@Data
public class CoordinateDto {

    private Double latitude;
    private Double longitude;
}
```

CreatePlanRequest

```
@Data
public class CreatePlanRequest {

    private String planName;

    private Double speed;

    private LocalDateTime startTime;

    private List<CoordinateDto> routePoints;
}
```

PlanResponse

```
@Data
@Builder
public class PlanResponse {

    private Long planId;

    private String planName;

    private Double speed;

    private LocalDateTime startTime;
}
```

Entities

Plan

Contains:

- Plan Name
- Speed
- Start Time
- Status
- Route Points

RoutePoint

Contains:

- Latitude
- Longitude
- Sequence Number
- Plan Mapping

Vehicle

Contains:

- Vehicle Name
- Vehicle Type
- Fuel Capacity

- Fuel Remaining
- Current Latitude
- Current Longitude
- Current Plan Id
- Current Point Index
- Simulation Started

Repositories

PlanRepository

```
public interface PlanRepository extends JpaRepository<Plan, Long> {
}
```

RoutePointRepository

```
public interface RoutePointRepository
        extends JpaRepository<RoutePoint, Long> {

    List<RoutePoint>
    findByPlanIdOrderBySequenceNo(Long planId);
}
```

VehicleRepository

```
public interface VehicleRepository
        extends JpaRepository<Vehicle, Long> {

    List<Vehicle> findBySimulationStartedTrue();
}
```

Plan Creation

API:

POST /api/plans

Stores:

- Plan
- Route Points

Validation:

- Plan Name Mandatory
- Speed > 0
- Start Time Mandatory
- Minimum 7 Route Points

Vehicle Bootstrap

At application startup:

Creates:

- ARMY_VEHICLE
- TANK
- TRUCK

Fuel Capacity:

1000 Litres

Fuel Remaining:

1000 Litres

Scheduler Configuration

Main Class

```
@SpringBootApplication
@EnableScheduling
public class DemoApplication {
}
```

Simulation Architecture

Frontend

↓

POST /api/simulation/start

↓

Simulation Service

↓

Scheduler

↓

Vehicle Movement

↓

Database Update

SimulationStateService

Purpose:

Maintain global simulation state.

Methods:

```
start()

stop()

isRunning()
```

SimulationService

Methods:

```
void startSimulation();

void stopSimulation();
```

Simulation Controller

Start:

```
POST /api/simulation/start
```

Stop:

```
POST /api/simulation/stop
```

Vehicle Assignment Logic

For Demo:

Vehicle 1 → Plan 1

Vehicle 2 → Plan 2

Vehicle 3 → Plan 3

During Start:

```
vehicle.setCurrentPlanId(plan.getId());

vehicle.setCurrentPointIndex(0);

vehicle.setSimulationStarted(true);
```

Scheduler

Runs every second.

```
@Scheduled(fixedRate = 1000)
```

Fuel Logic

Requirement:

1 litre / minute

Scheduler:

1 second

Formula:

```
double fuelConsumed = 1.0 / 60.0;
```

Update:

```
vehicle.setFuelRemaining(
        Math.max(
                vehicle.getFuelRemaining()
                - fuelConsumed,
                0.0
        )
);
```

Vehicle Movement Logic

Current POC:

Point-to-Point movement.

Example:

Point1

↓

Point2

↓

Point3

↓

Point4

Every scheduler execution:

```
vehicle.setCurrentLatitude(
        point.getLatitude());

vehicle.setCurrentLongitude(
        point.getLongitude());

vehicle.setCurrentPointIndex(
        index + 1);
```

Route Completion

```
if(index >= points.size())
{
    vehicle.setSimulationStarted(false);
}
```

Fuel Exhaustion

```
if(vehicle.getFuelRemaining() <= 0)
{
    vehicle.setSimulationStarted(false);
}
```

Current Features Completed

✔ Create Plan

✔ Save Route Points

✔ Bootstrap Vehicles

✔ Start Simulation

✔ Stop Simulation

✔ Scheduler

✔ Fuel Consumption

✔ Point-to-Point Vehicle Movement

✔ Database Updates

Remaining Features (Future)

1. SSE Events

```
Vehicle Position Streaming
```

1. Time Based Mode

```
1 Second
1 Minute
```

1. Event Based Mode

```
Move on Event
```

1. Forward Playback
2. Backward Playback
3. Time Slider
4. Smooth Interpolation
5. Replay Functionality
6. Route History
7. Simulation Reports

Run Commands

Build

```
mvn clean install
```

Run

```
mvn spring-boot:run
```

Test Flow

Step 1

Create:

PLAN_A

PLAN_B

PLAN_C

Step 2

POST

```
/api/simulation/start
```

Step 3

Observe vehicle table updates.

Step 4

POST

```
/api/simulation/stop
```

Simulation stops successfully.